

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet

Docente: Dr. Mauro Dragoni

Dipartimento di Ingegneria e Scienze dell'Informazione
Anno Accademico 2025/2026

materiale di riferimento

- Android Developer, sito web di riferimento dei realizzatori di Android
<https://developer.android.com/>
- John Horton. Android Programming for Beginners

perche' la programmazione mobile e' diversa?

- Display che variano da piccoli smartphone a smart TV di 60+ pollici.
- Diverse versioni di sistemi operativi (multiple APK)
- Scarse risorse (memoria, disco, ...)
- Condizioni mutabili della connettivita' (GSM, WiFi)
- Gestione del trasferimento di dati: costi, velocita', latenza
- Batteria
- Priorita' (cosa succede se arriva una telefonata mentre usano la vostra app?)
- Diverse interazioni utente (tastiera, touch, camera, ...)
- Dispositivi e sensori (accelerometro, GPS, camera, audio, microfono)
- Speech APIs
- Comunicazione tra diverse applicazioni
- Sicurezza
- Modello di sviluppo (compilazione cross-piattaforma)
- Modello di distribuzione (store)

filosofia di design

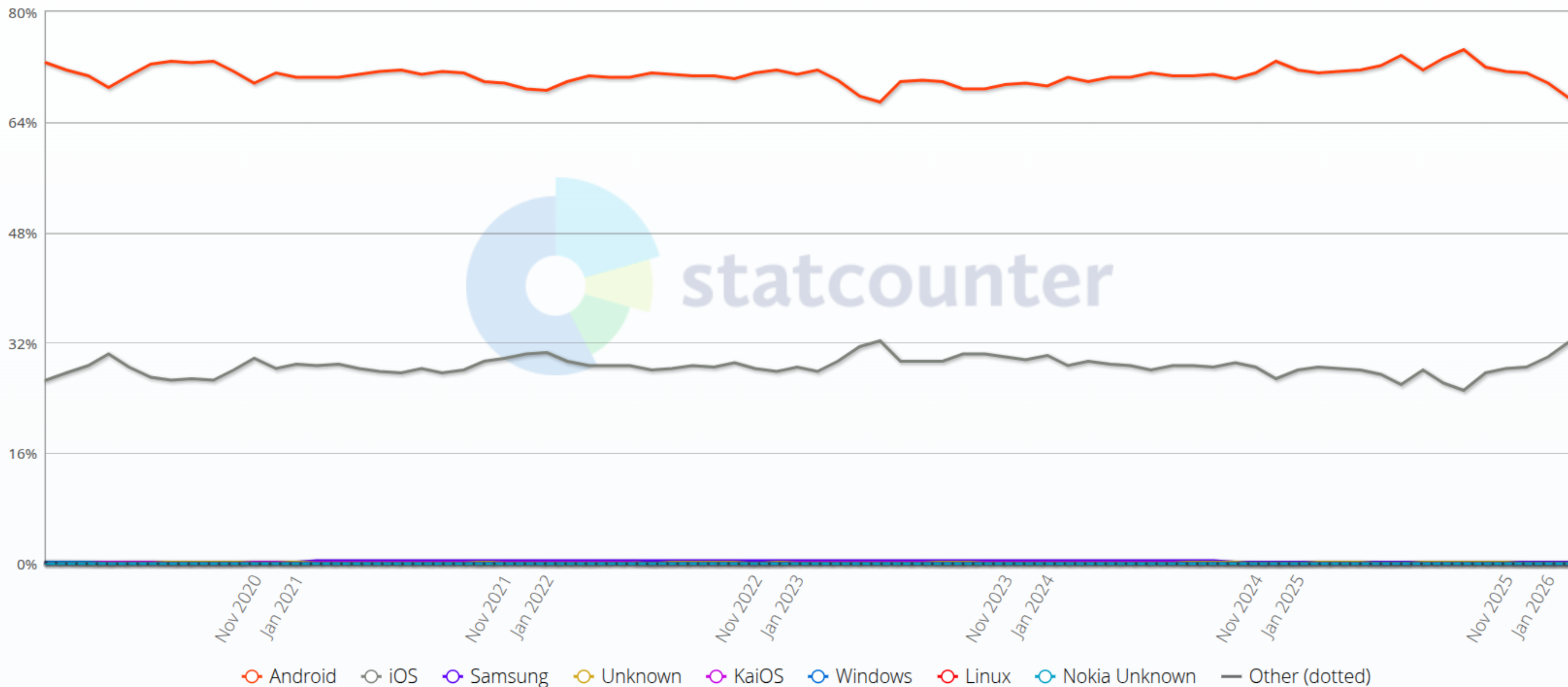
Le applicazioni **DOVREBBERO** essere:

- Veloci
 - nonostante vincoli del tipo: < 200 MB RAM, processore lento, ...
- Responsive
 - le applicazioni devono rispondere all'azione utente entro 5 secondi
- Sicure
 - le applicazioni devono dichiarare i permessi già a livello di manifesto
- Senza soluzione di continuita'
 - persistenza dei dati, gestione della sospensione dei servizi
 - il sistema operativo puo' terminare processi in background se necessario

market share

Mobile & Tablet Operating System Market Share Worldwide

Jan 2020 - Feb 2026



cos'è Android?

- Android è un insieme di componenti software, comprendente:
 - un sistema operativo,
 - un middleware, e
 - un insieme di applicazioni basilari
- Android è usualmente utilizzato in smartphones e tablets ma potrebbe essere esteso a supportare qualsiasi dispositivo, ipoteticamente anche un PC.
- Android è sia uno stack open-source per dispositivi mobili, ma è anche un progetto open-source guidato da Google.

la licenza di Android

							
Type		Permissive	Permissive	Permissive	Copyleft	Copyleft	Copyleft
Provides copyright protection		✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE
Can be used in commercial applications		✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE	✓ TRUE
Provides an explicit patent license		✓ TRUE	✗ FALSE	✗ FALSE	✗ FALSE	✗ FALSE	✗ FALSE
Can be used in proprietary (closed source) projects		✓ TRUE	✓ TRUE	✓ TRUE	✗ FALSE	✗ FALSE partially	✗ FALSE for web
Popular open-source and free projects		Kubernetes Swift Firebase	Django React Flutter	Angular.js jQuery, .NET Core Laravel	Joomla Notepad++ MySQL	Qt SharpDevelop	SugarCRM Launchpad

sviluppo e governance

- In ciascun momento esiste una **current release** della piattaforma Android
- Tutti i produttori di device e i vari contributors possono lavorare sulla current release per correggere bugs, promuovere nuovi dispositivi, sperimentare nuove funzionalità, ecc.
- In parallelo, Google lavora internamente sulla versione successiva in accordo con i bisogni del prodotto e gli obiettivi aziendali. Solitamente Google prende come riferimento un dispositivo partner le cui specifiche sono in linea con la direzione di sviluppo di Android.
- Quando la “**n+1**” versione è pronta, viene pubblicata e diventa la nuova **current release**.

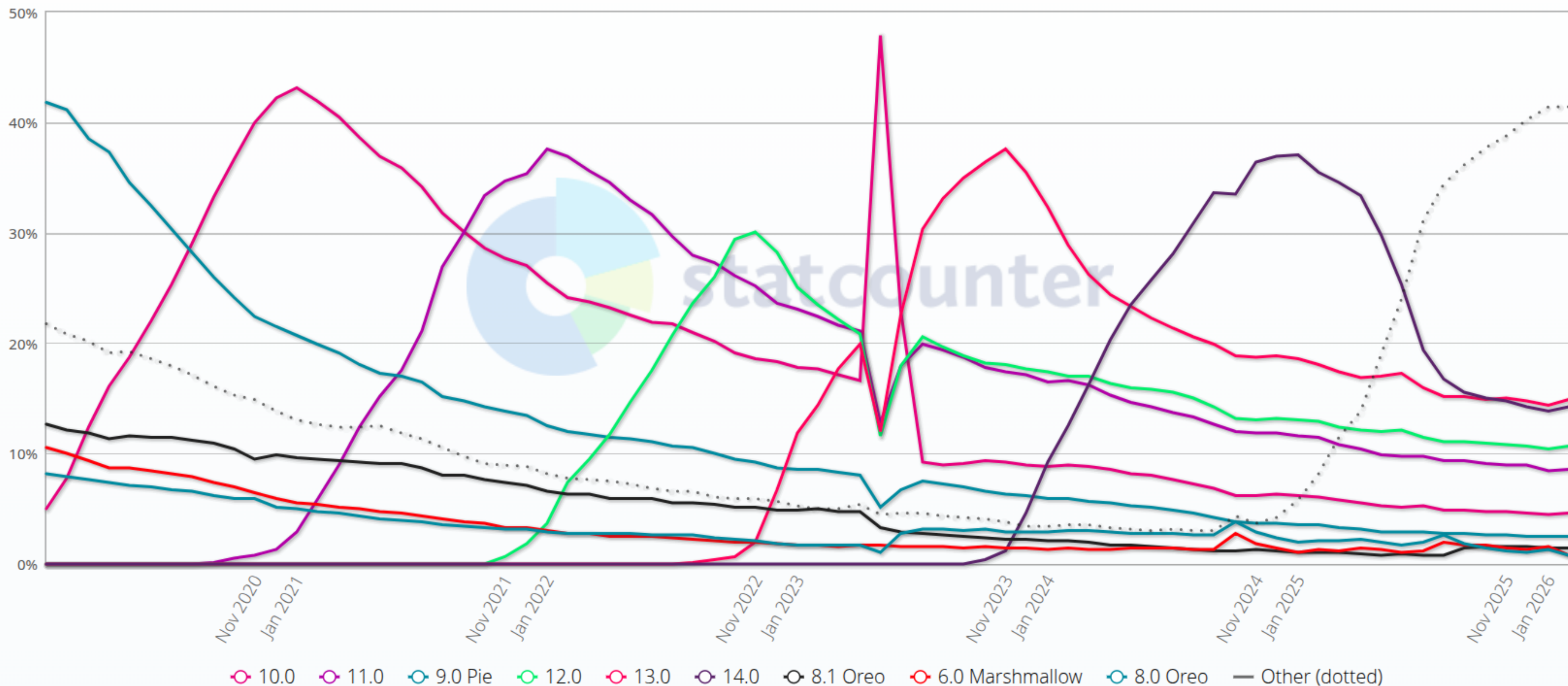
un po' di statistiche: distribuzioni

	Platform Version	API Level	% of Distribution	Cumulative Distribution
5	Lollipop	21	0.1%	100.0%
5.1	Lollipop	22	0.3%	99.9%
6	Marshmallow	23	0.4%	99.6%
7	Nougat	24	0.4%	99.2%
7.1	Nougat	25	0.4%	98.8%
8	Oreo	26	0.8%	98.4%
8.1	Oreo	27	2.3%	97.6%
9	Pie	28	4.5%	95.3%
10	Q	29	7.8%	90.8%
11	R	30	13.7%	83.0%
12	S	31	11.4%	69.3%
13	T	33	13.9%	57.9%
14	U	34	17.2%	44.0%
15	V	35	19.3%	26.8%
16	B	36	7.4%	7.5%

un po' di statistiche: distribuzioni

Mobile & Tablet Android Version Market Share Worldwide

Jan 2020 - Feb 2026



un po' di statistiche: distribuzioni

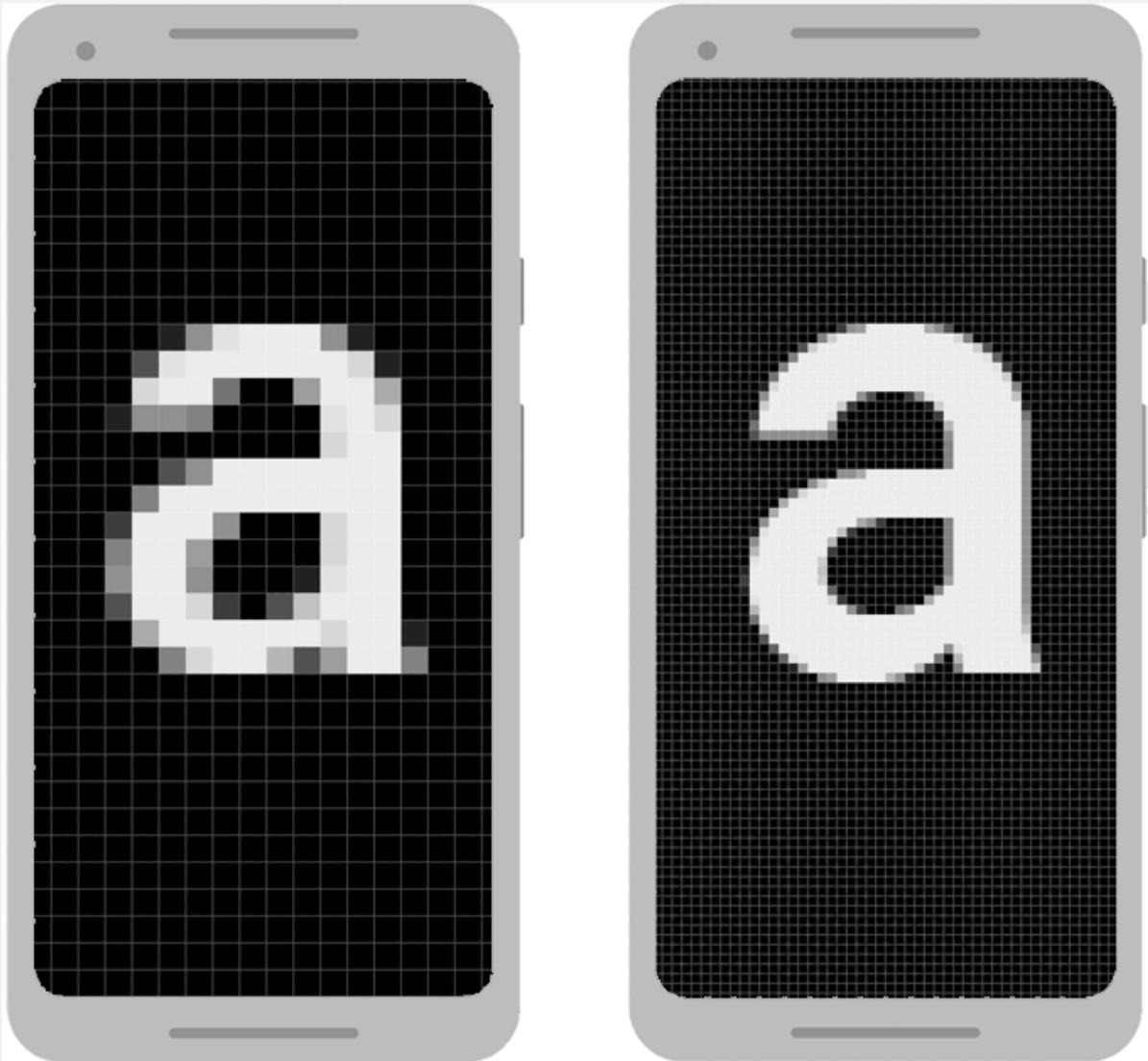
```
<manifest xmlns:android="http://schemas.android.com/apk/res/android" ... >
    <uses-sdk android:minSdkVersion="4" android:targetSdkVersion="15" />
    ...
</manifest>
```

```
private void setUpActionBar() {
    // Make sure we're running on Honeycomb or higher to use ActionBar APIs
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.HONEYCOMB) {
        ActionBar actionBar = getActionBar();
        actionBar.setDisplayHomeAsUpEnabled(true);
    }
}
```

un po' di statistiche: display

	ldpi	mdpi	tvdpi	hdpi	xhdpi	xxhdpi	Total
Small					0.5%		0.5%
Normal			0.2%	2.8%	45.2%	23.7%	71.9%
Large		0.9%	4.8%	0.9%	10.4%	0.9%	17.9%
Xlarge		4.4%	0.1%	4.4%	0.8%		9.7%
Total	0.0%	5.3%	5.1%	8.1%	56.9%	24.6%	

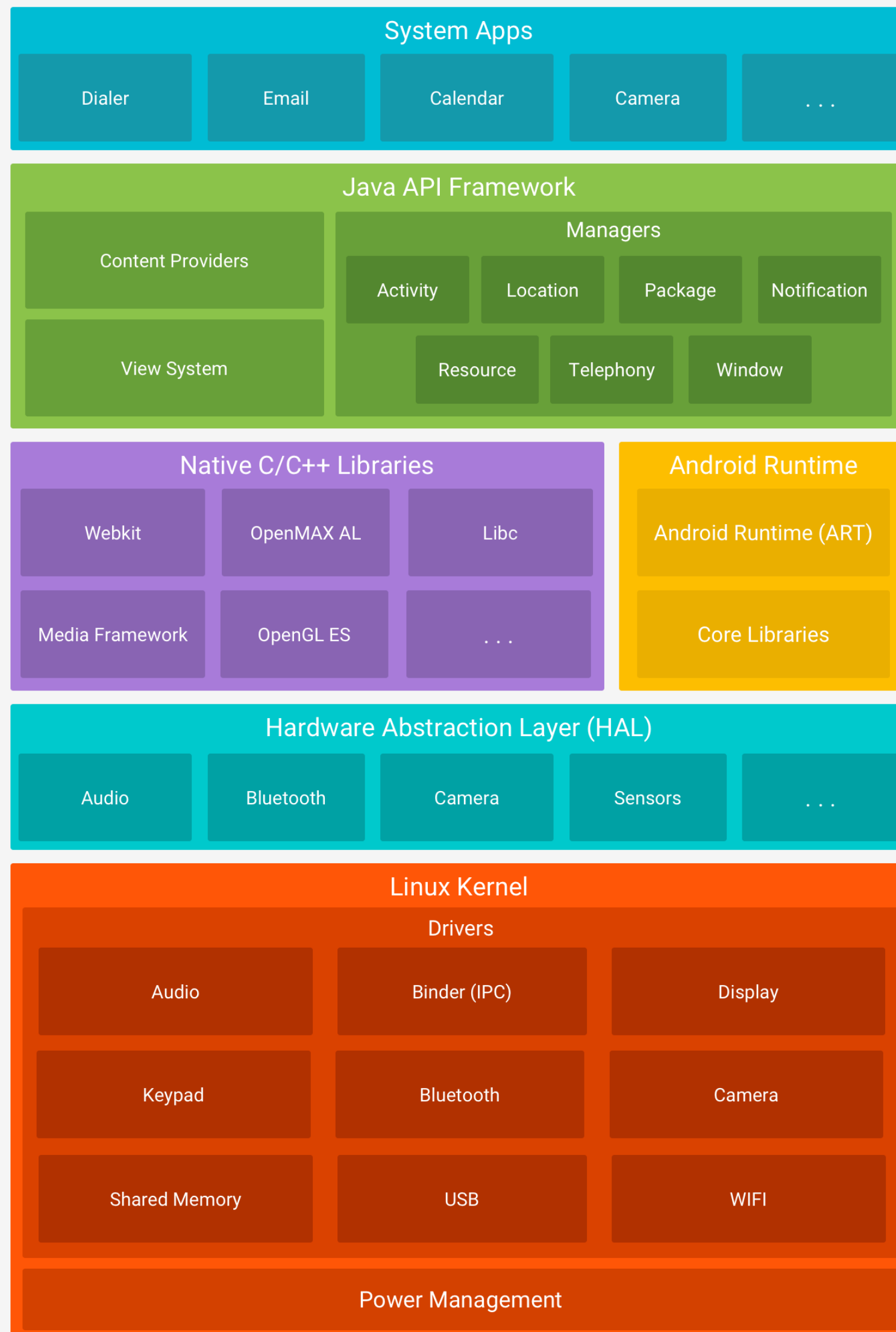
Data collected during a 7-day period ending on November 24, 2025.
Any screen configurations with less than 0.1% distribution are not shown.



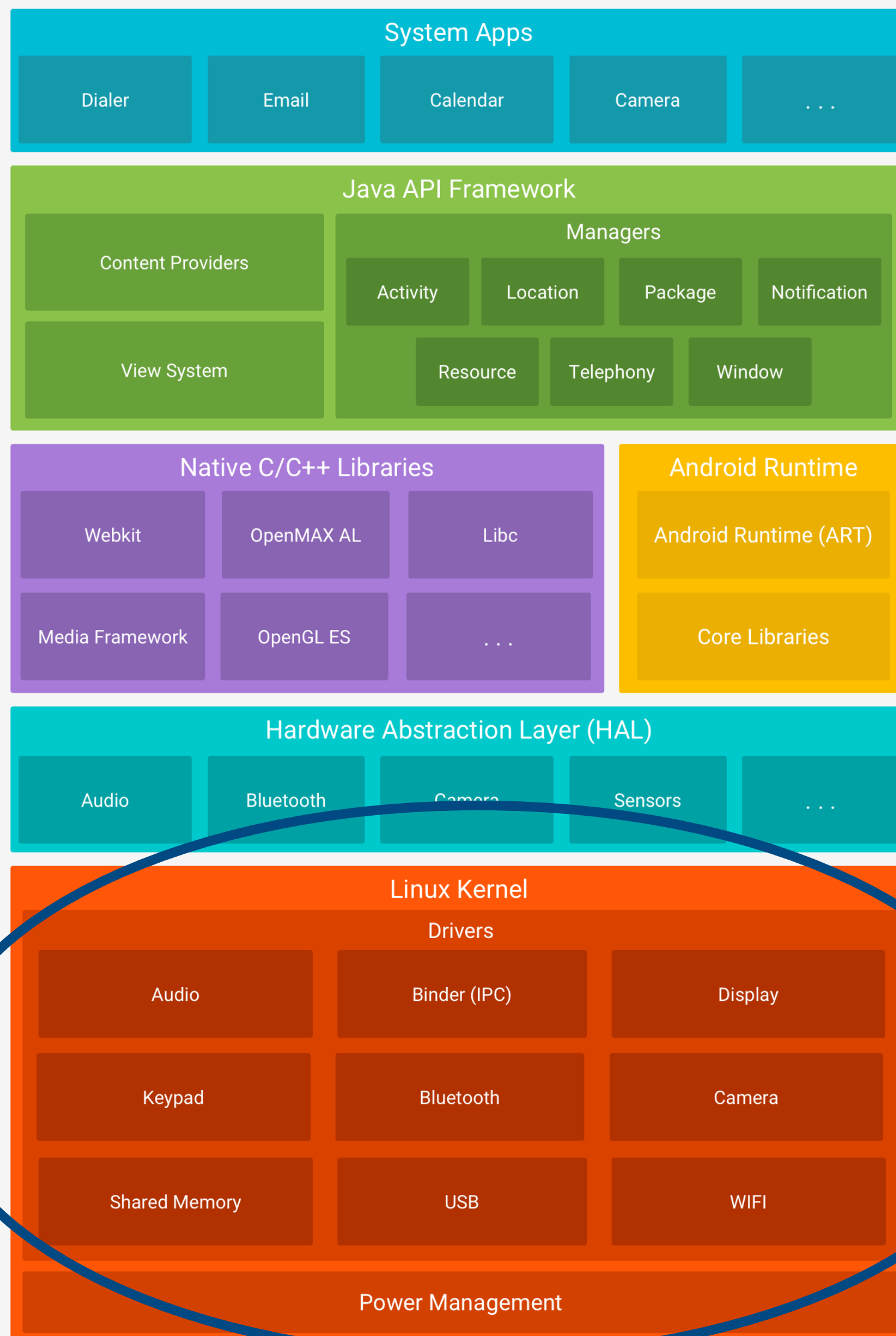
cosa supporta Android

- User Interface
 - IO widgets (buttons, textboxes, lists)
 - Images
 - 2D/3D drawing
- Database
- Integrated browser
- Media support (audio, video, images; camera)
- Application framework lifecycle
- Connectivity (bluetooth, wi-fi, EDGE, 3G)
- Sensors (GPS/Geo-location, accelerometer, compass)
- GSM Telephony (call – sms)
- Google Maps
- Multiple processes
 - Managed by Android Dalvik VM
 - Background Services
 - Interprocess communications (e.g. Intents)
- Rich development environment including a device emulator, debugging tools, memory and performance profiling, Eclipse plug in

Android stack



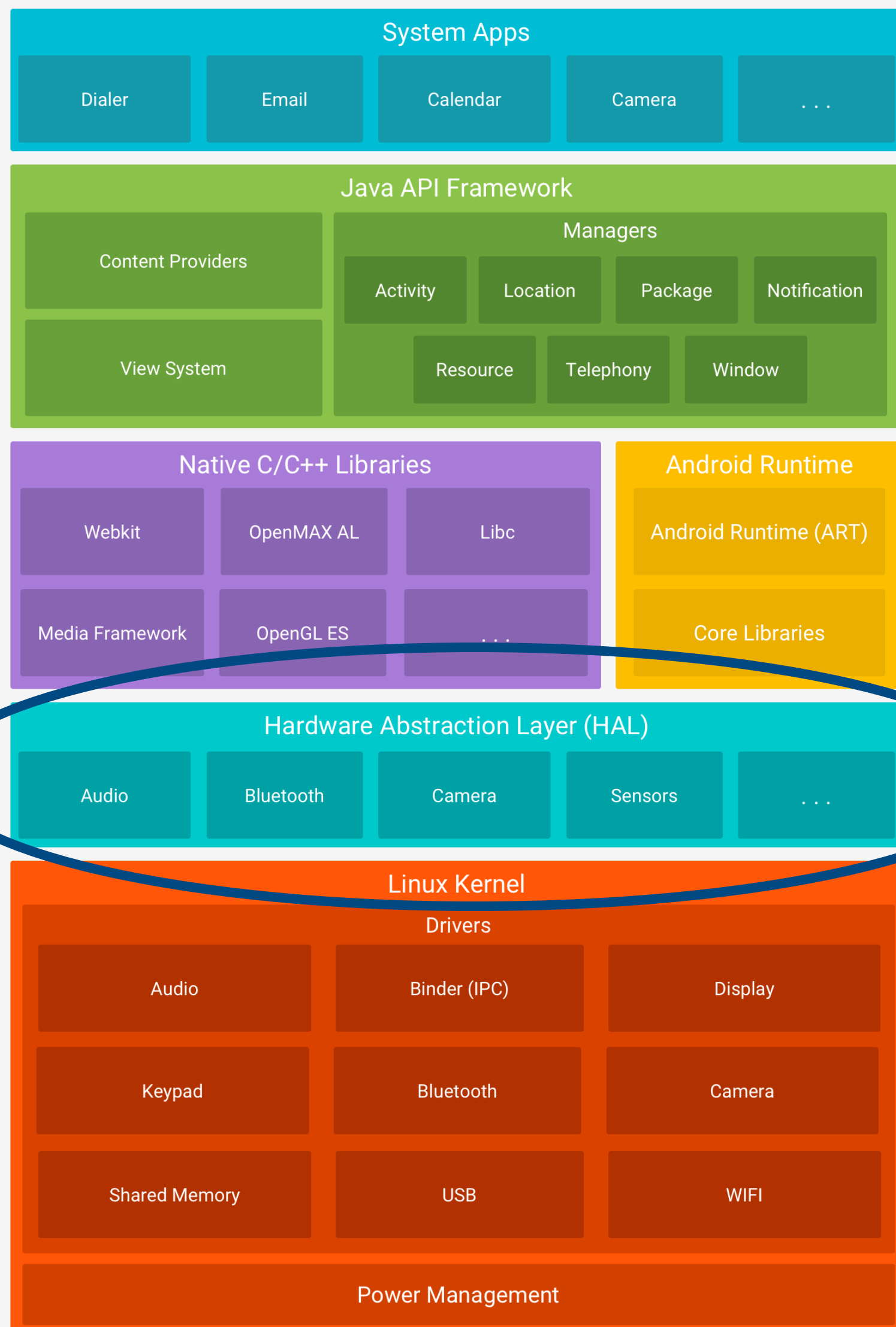
Android stack



Il fondamento della piattaforma Android e' il kernel Linux. Ad esempio, Android Runtime (ART) si affida al kernel Linux per funzionalita' sottostanti come il threading e la gestione della memoria di basso livello.

L'uso di un kernel Linux consente ad Android di sfruttare le principali funzionalita' di sicurezza e consente ai produttori di dispositivi di sviluppare driver hardware per un noto kernel.

Android stack

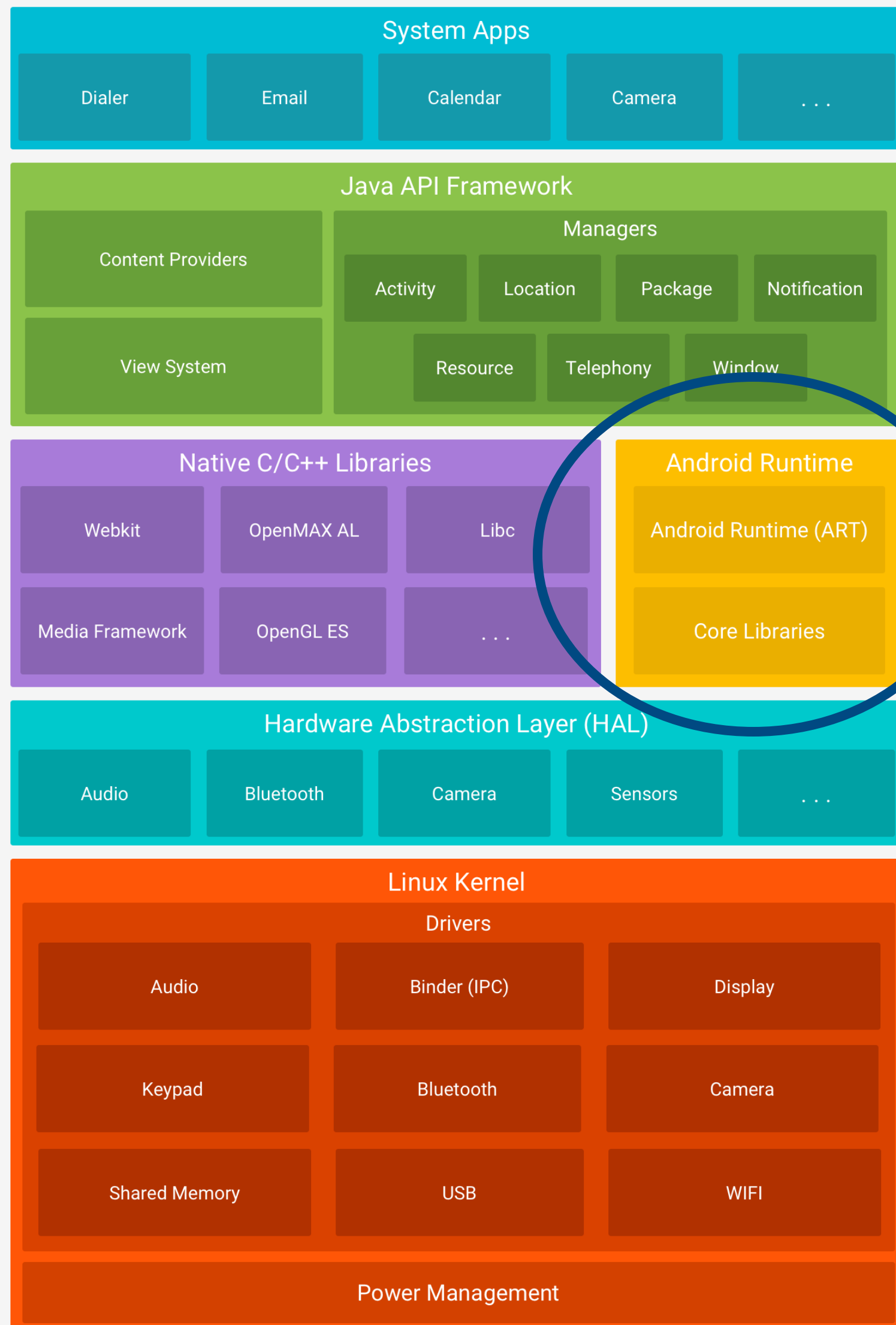


Il livello di astrazione hardware (HAL) fornisce le interfacce standard che espongono le capacità hardware del dispositivo al framework Java (API) di livello superiore.

L'HAL è costituito da più moduli di libreria, ognuno dei quali implementa un'interfaccia per un tipo specifico di componente hardware, come il modulo videocamera o bluetooth.

Quando un'API del framework effettua una chiamata per accedere all'hardware del dispositivo, il sistema Android carica il modulo della libreria per quel componente hardware.

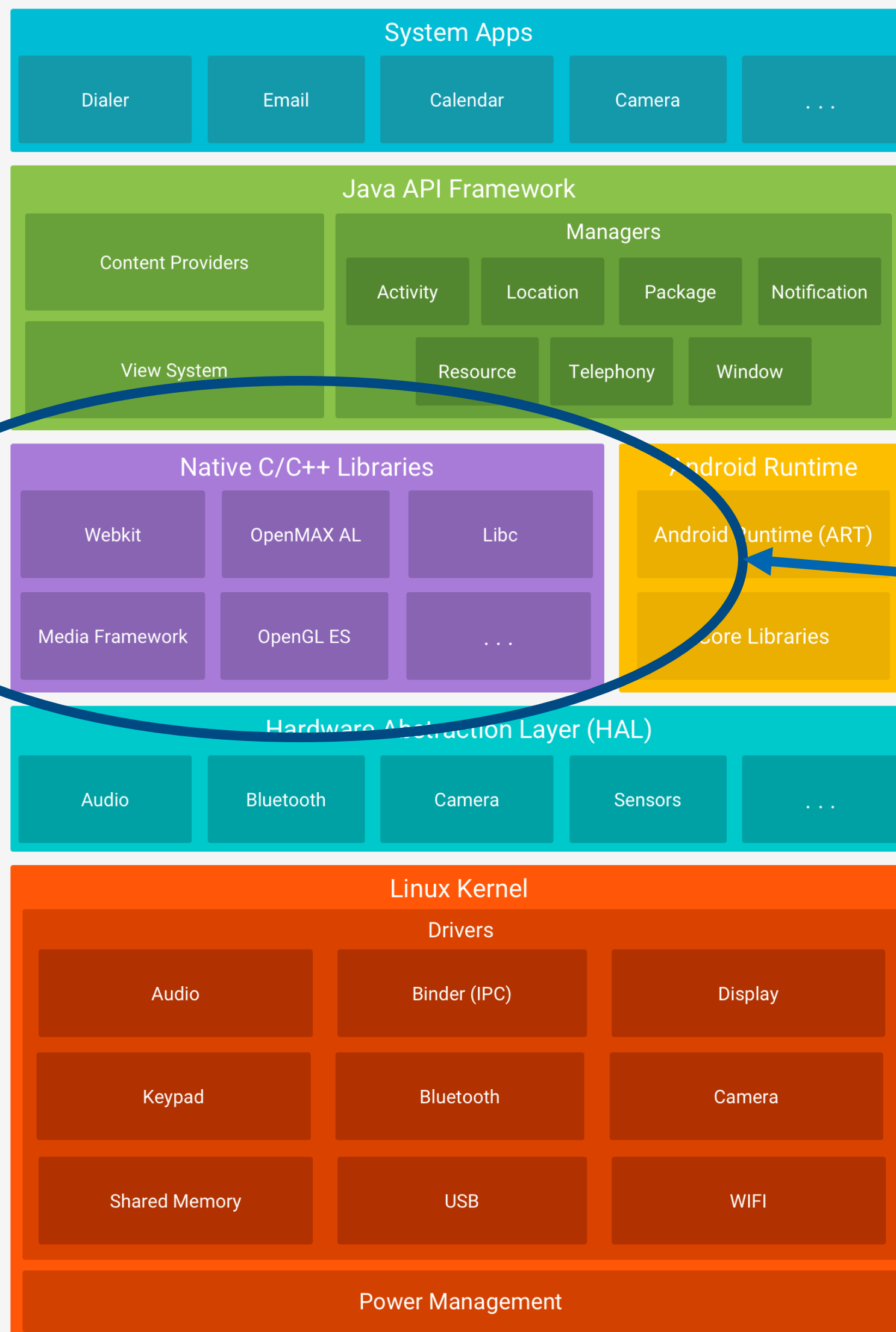
Android stack



Ambiente di runtime contenente la JVM incaricata di eseguire le applicazioni.

(vediamo i dettagli dopo)

Android stack



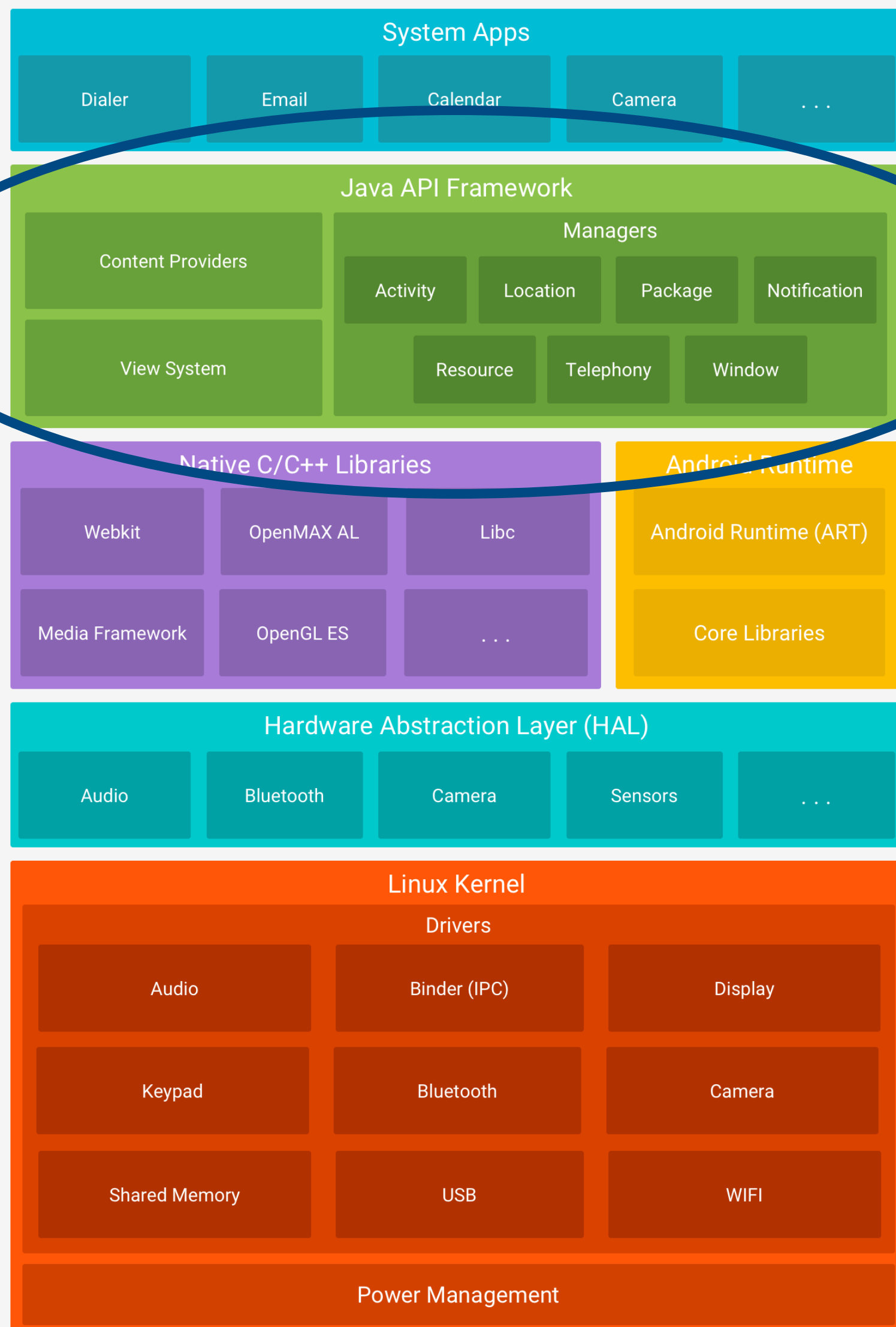
Molti componenti e servizi del sistema Android di base, come ART e HAL, sono costruiti da codice nativo che richiede librerie native scritte in C e C ++.

La piattaforma Android fornisce API per esporre le funzionalità di alcune di queste librerie native alle applicazioni.

Ad esempio, puoi accedere a OpenGL ES tramite le Java OpenGL API del framework Android per aggiungere supporto per il disegno e la manipolazione di grafica 2D e 3D nella propria.

Se si sta' sviluppando un'applicazione che richiede codice C o C ++, si può utilizzare Android NDK per accedere ad alcune di queste librerie della piattaforma nativa direttamente dal proprio codice nativo.

Android stack

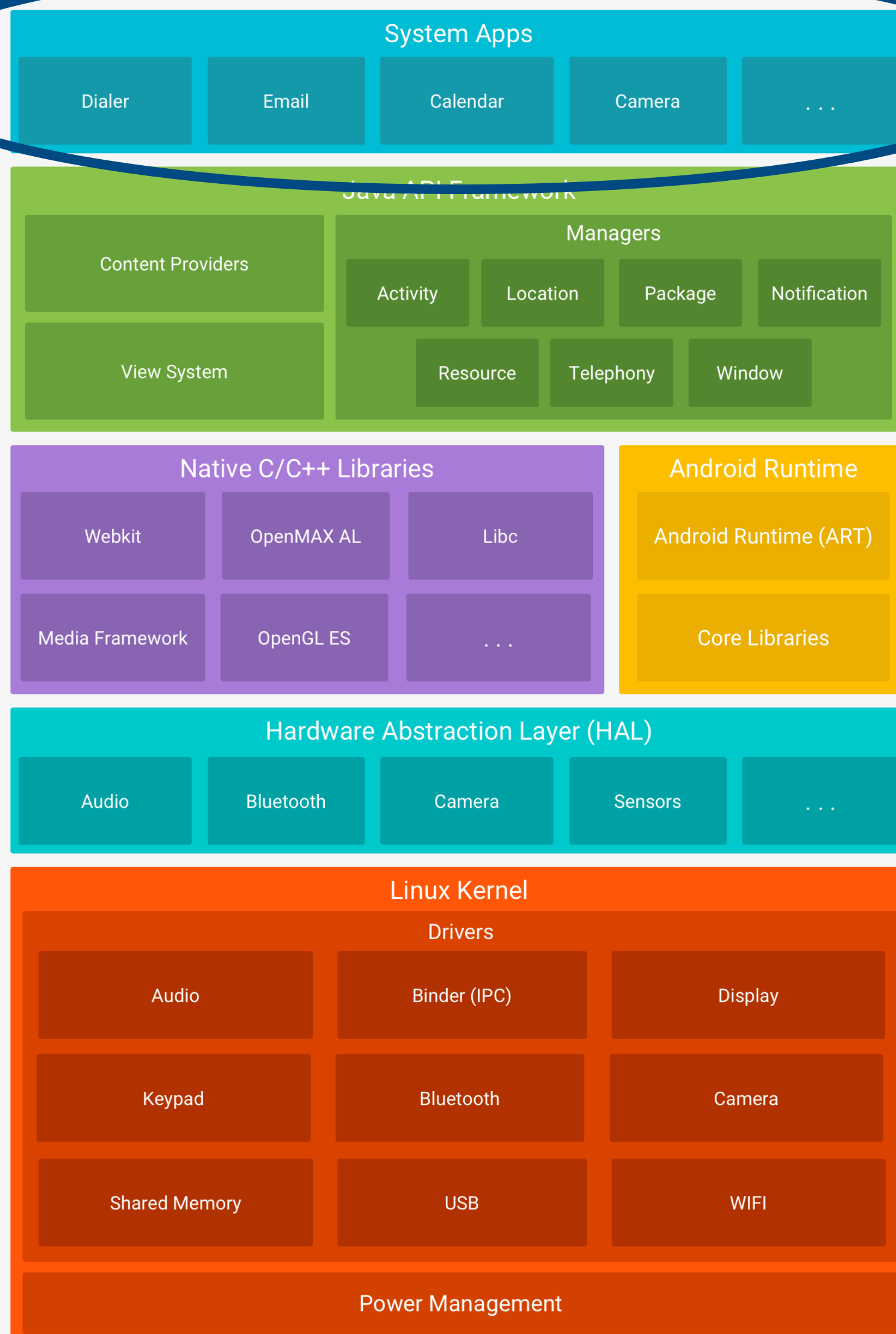


Queste API costituiscono i mattoni necessari per creare applicazioni Android semplificando il riutilizzo del core di Android e lo sviluppo modulare. Includono quanto segue:

- Un **View System** ricco ed estensibile che si può utilizzare per creare l'interfaccia utente di un'applicazione, inclusi elenchi, griglie, caselle di testo, pulsanti e persino un browser Web incorporabile.
- Un **Resource Manager** che fornisce accesso a risorse (non codice) come localizzazione, grafica e file di layout.
- Un **Notification Manager** che consente a tutte le applicazioni di visualizzare avvisi personalizzati nella barra di stato.
- Un **Activity Manager** che gestisce il ciclo di vita delle applicazioni e fornisce uno stack di navigazione comune.
- Un **Content Provider** che consente alle applicazioni di accedere ai dati di altre applicazioni. Esempio l'applicazione Contatti.

Gli sviluppatori hanno pieno accesso alle stesse API framework utilizzate dalle applicazioni di sistema di Android.

Android stack



Android viene fornito con una serie di applicazioni principali per e-mail, messaggi SMS, calendari, navigazione in Internet, contatti e altro. Le applicazioni incluse con la piattaforma non hanno uno «status» speciale rispetto alle applicazioni che l'utente sceglie di installare.

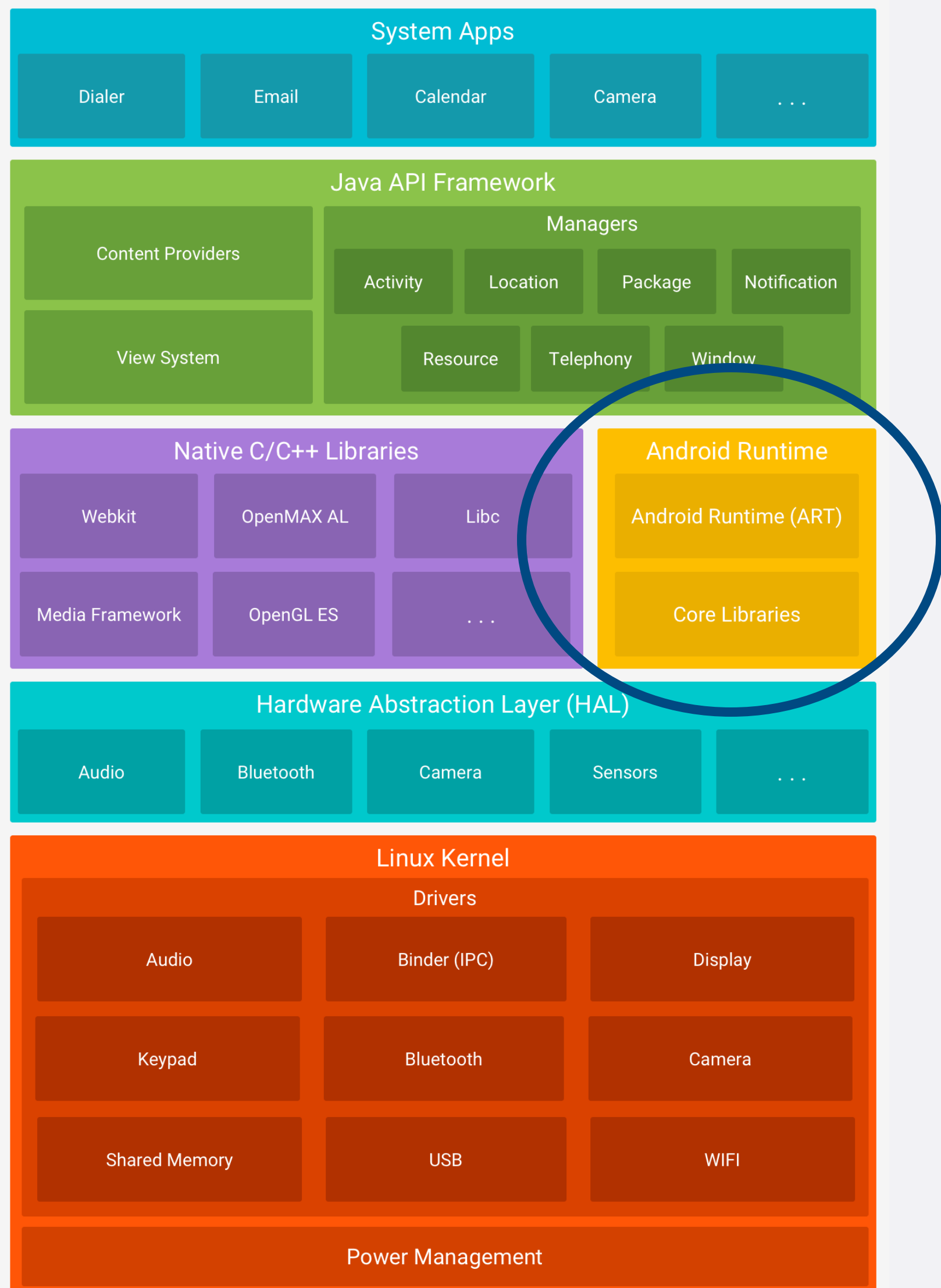
Pertanto, un'applicazione di terze parti può diventare il browser Web predefinito, il servizio di messaggistica SMS o persino la tastiera predefinita (si applicano alcune eccezioni, ad esempio l'applicazione «Impostazioni del sistema»).

Le applicazioni di sistema funzionano sia come applicazioni per gli utenti sia per fornire funzionalità chiave a cui gli sviluppatori possono accedere dalle proprie applicazioni. Ad esempio, se la propria applicazione desidera recapitare un messaggio SMS, non è necessario creare tale funzionalità ex novo ma si può invocare qualunque applicazione SMS già installata per recapitare un messaggio al destinatario specificato.

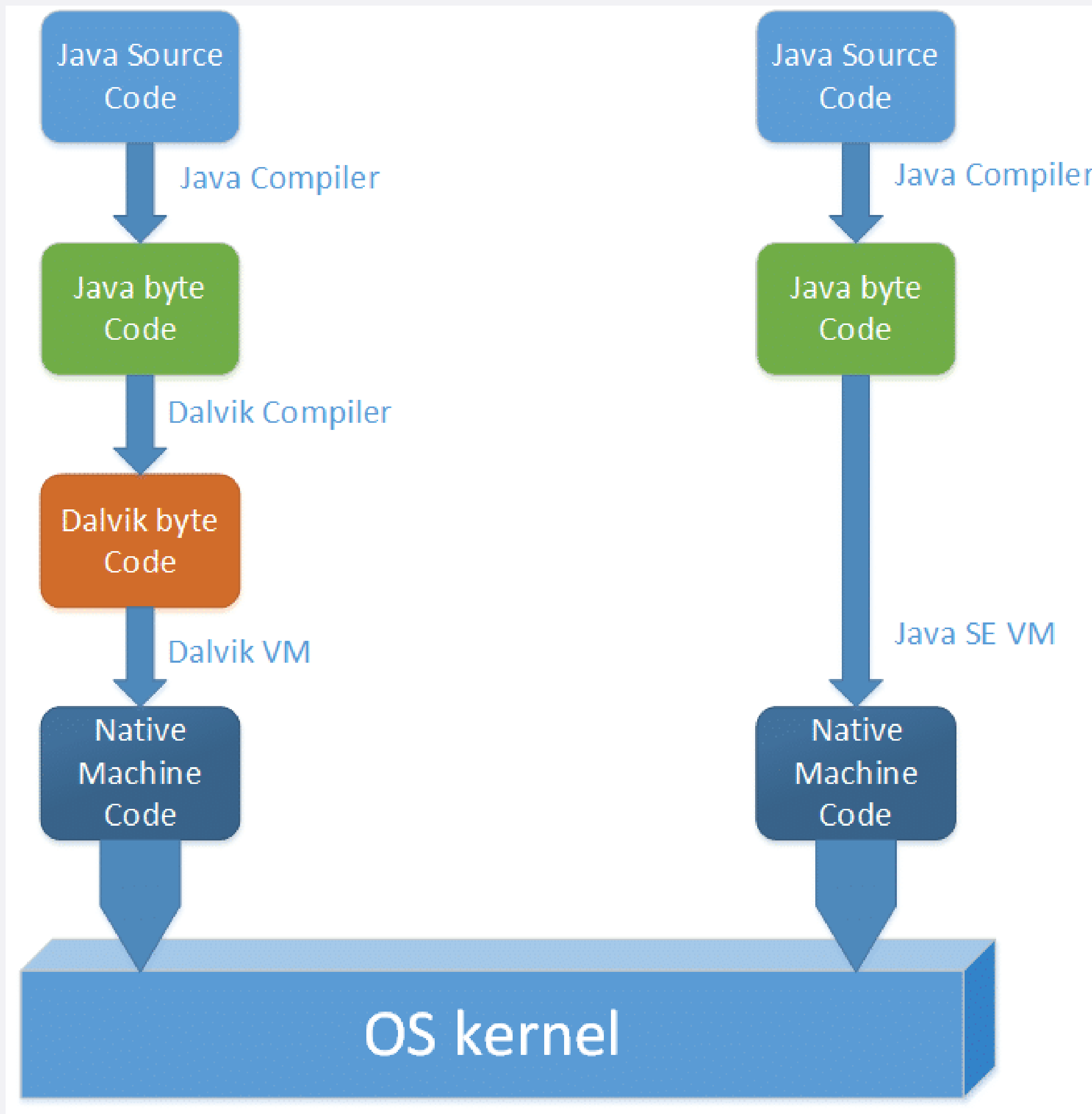
sistema operativo e librerie

- Il sistema operativo sottostante è una distribuzione, opportunamente ridotta di Linux
- Un certo numero di librerie di base sono state inserite, per supportare alcune features fondamentali:
 - Handset layouts
 - Storage
 - Connectivity
 - Messaging
 - Multiple language support
 - Web browser
 - Java support
 - Media support
 - Streaming media support
 - Additional hardware support
 - Multi-touch
 - Bluetooth
 - Video calling
 - Multitasking
 - Voice based features
 - Tethering
 - Screen capture

riprendiamo dalla virtual machine



Java vs. Dalvik



Dalvik Virtual Machine

- All'interno di ogni dispositivo Android e' presente una virtual machine, denominata Dalvik
 - Dalvik e' una macchina virtuale open source in grado di eseguire bytecode, in maniera simile alla Java Virtual Machine della Sun
 - Il bytecode e' denominato dex (Dalvik executable)
- Dalvik e' ottimizzata per macchine dalla ridotta memoria
 - Gestisce i thread, con alcune limitazioni
 - Non gestisce le eccezioni
- Ogni applicazione su Android e' vista come un diverso user, con un proprio processo, una propria zona dati e una propria istanza di Dalvik virtual machine.
- Dalvik e' stata integrata/sostituita da ART

ART Virtual Machine

- Nuova virtual machine
 - In Android 4.4 Kit Kat si affianca alla Dalvik
 - A partire da Android 5 Lollipop e' l'unica VM
- ART e' basata su tecnologia AOT (ahead-of-time)
 - Il codice compilato dell'applicazione e' direttamente eseguibile
- Cio' riduce notevolmente i tempi di esecuzione rispetto ad un compilatore JIT (Just-In-Time) come Dalvik
 - Di conseguenza c'e' anche un corrispondente risparmio energetico
 - Aumenta, invece, il consumo di spazio sulla memoria del dispositivo
- Salvo eccezioni, e' garantita la compatibilita' delle applicazioni (il dex compilato e' anche l'input per la ART)

riassumendo: i tre blocchi principali

- **Dispositivo Hardware:** Android funziona su una vasta gamma di configurazioni hardware tra cui smartphone, tablet, ecc.. Android e' agnostico rispetto al processore, ma sfrutta alcune funzionalità di sicurezza specifiche dell'hardware (ad es. ARM).
- **Sistema Operativo Android:** Il sistema operativo principale e' basato sul kernel Linux. Tutte le risorse del dispositivo sono accessibili tramite il sistema operativo.
- **Android Application Runtime:** Le applicazioni Android sono spesso scritte in Java ed eseguite nella Dalvik VM. Tuttavia, molte applicazioni, inclusi i principali servizi e applicazioni Android, sono applicazioni native o includono librerie native. Sia Dalvik che le applicazioni native vengono eseguite nello stesso ambiente di sicurezza, nella Sandbox dell'applicazione. Le applicazioni ottengono una parte dedicata del file system in cui possono scrivere dati privati, inclusi database e file non elaborati.

Android non segue uno standard Java

- Distribuzioni Standard Java:
 1. Java Standard Edition: utilizzato per lo sviluppo su applicazioni desktop di base.
 2. Java Enterprise Edition (aka J2EE or JavaEE): utilizzato per lo sviluppo di applicazioni aziendali.
 3. Java Micro Edition (aka J2ME or JavaME): Java for embedded systems.
- Il set di librerie Java di Android e' il piu' vicino a Java Standard Edition. La differenza principale e' che le librerie Java per l'interfaccia utente (AWT e Swing) sono state rimosse e sostituite con librerie di interfaccia utente specifiche per Android.
- Android aggiunge anche alcune nuove funzionalita' a Java standard, pur supportando la maggior parte delle funzionalita' standard di Java.

macroclassificazione delle tipologie di applicazioni

- **Classica:** tutte le interfacce, servizi, grafica, database, ecc. sono sviluppate all'interno dell'applicazione Android.
- **Web-based:** viene creata una sola interfaccia contenente una WebView e si sviluppa tutto l'applicativo all'interno di un server web esterno.
- **Integrazione con terze parti:** si sfrutta l'architettura di altre applicazioni per sviluppare la propria (es. bot telegram)

| l'ambiente di sviluppo

- Due principali alternative:
 - **Apache Studio**
 - **Eclipse con ADT**

| l'ambiente di sviluppo: Android Studio

- Deriva dalla famiglia di IntelliJ. Google contribuisce al supporto.
- Utilizzo di Gradle per gestire il file di build e le configurazioni.
- Altre caratteristiche messe a disposizione da Android Studio sono:
 - **editor per layout** visuale usabile in modalita' drag and drop;
 - **supporto a ProGuard** (code and resource shrinking, obfuscation, optimization) e preparazione del pacchetto di installazione;
 - accesso a **SDK Manager** per la personalizzazione dell'SDK scaricato e AVD (Android Virtual Device) Manager per la gestione degli emulatori;
 - **inline debugging**, funzionalita' che rende più immediata l'ispezione del codice durante il debug, affiancando alle righe di linguaggio Java i valori ed i riferimenti collegati agli oggetti;
 - **monitoraggio delle risorse** di memoria e della CPU utilizzate dall'applicazione, per seguire l'allocazione dinamica di oggetti ed effettuare dump della memoria heap, da analizzare successivamente.

l'ambiente di sviluppo: Android Studio

- Richiede un minimo di requisiti hardware per un funzionamento ottimale:
 - sistemi operativi sufficientemente moderni (per Windows almeno la versione 7, per Mac minimo OS X 10.10 Yosemite, per Linux si raccomanda un ambiente grafico KDE o GNOME);
 - non meno di 4 GB di RAM, possibilmente 8;
 - almeno 2 GB di spazio disco libero ma ne sono raccomandati 4.

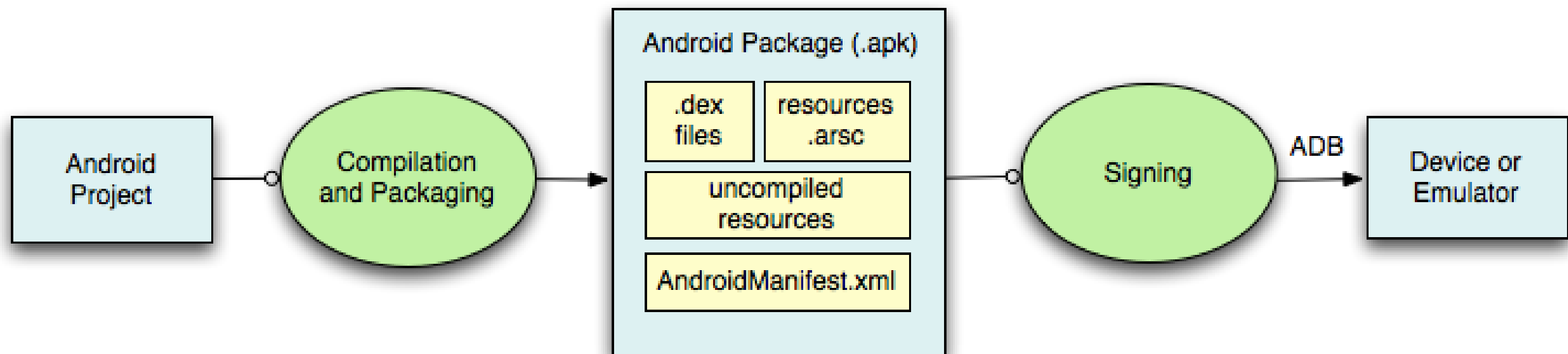
| l'ambiente di sviluppo: Eclipse con ADT

- ADT (Android Development Toolkit) e' un'estensione di eclipse totalmente dedicata ad Android.
- Integra nell'ambiente di sviluppo Eclipse tutti gli strumenti necessari allo sviluppo e all'esecuzione di un'applicazione Android
 - In particolare, rende possibile l'utilizzo di quasi tutte le utility disponibili a linea di comando
- Negli ultimi anni, l'uso di Eclipse ADT e' sempre meno diffuso, in favore di Android Studio (che rimane la scelta consigliata)

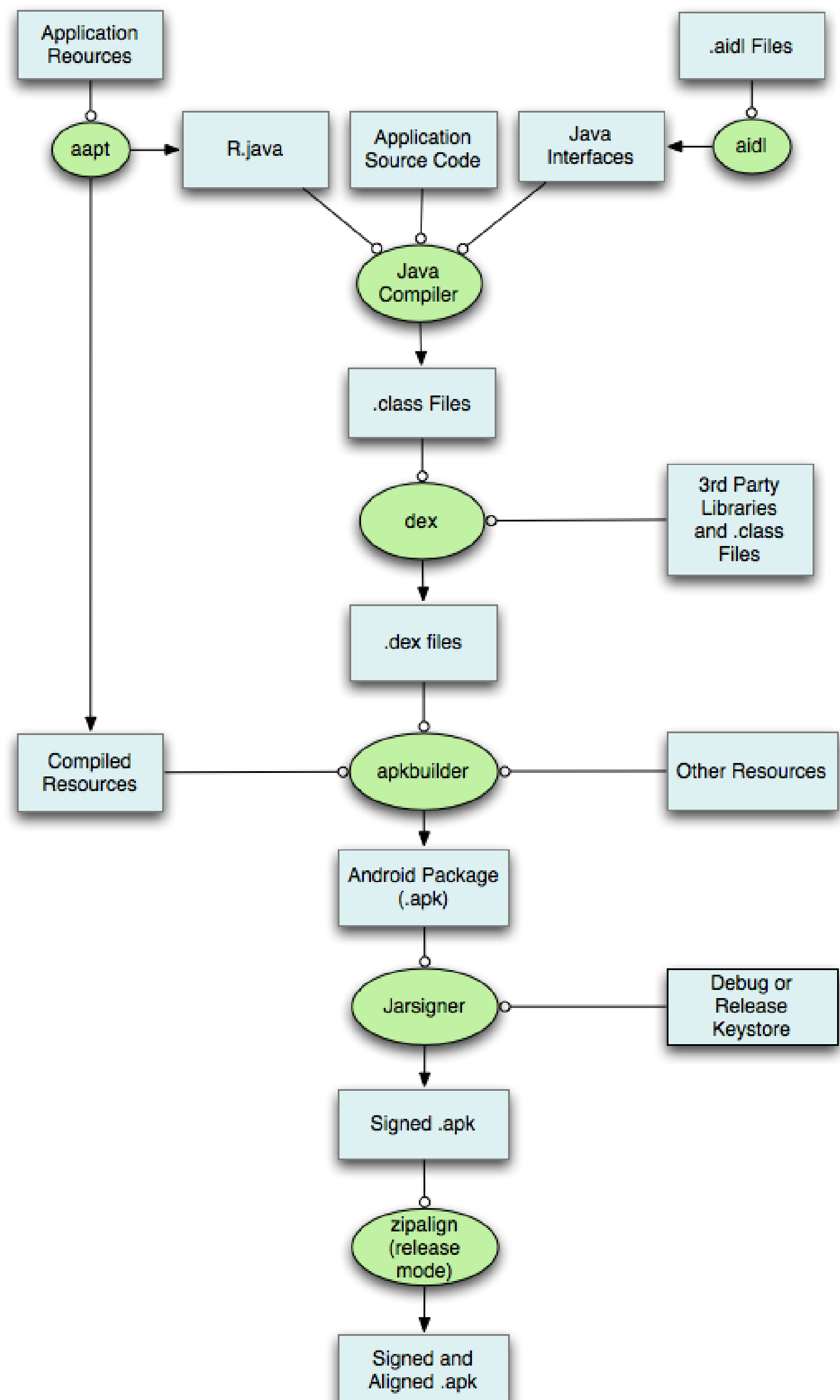
Android Studio vs. Eclipse ADT

- Principali differenze:
 - Più semplice gestione delle librerie (in ADT spesso è necessario includerle come sottoprogetti)
 - Più semplice gestione delle dipendenze (grazie a gradle utilizzato al posto di ant)
 - Maggiore quantitativo di funzioni utili già presenti nell'IDE (in Eclipse si deve installarle come plugin di terze parti)

processo di build



- Aapt: Android Asset Packaging Tool
 - Legge gli xml e genera R.java
- Aidl: Android Interface Definition Language
 - Aidl converte interfacce di servizi .aidl in interfacce Java
- Tutto il codice java e' compilato generando bytecode .class
- Dex converte i .class in file dex eseguibili da Dalvik/ART (e include eventuali librerie)
- Apkbuilder comprime e impacchetta i .dex e le risorse (grafiche, etc.) in un unico file .apk
- Jarsigner permette di inserire una firma nel .apk
- Zipalign consente di ottimizzare le risorse di memoria utilizzata dall'applicazione in un dispositivo



signing

- Ogni applicazione Android ha bisogno di una firma privata, che identifichi l'autore, per poter essere pubblicata
 - La firma serve per garantire fiducia nell'applicazione
 - La firma e' messa direttamente dall'autore stesso
 - La firma puo' essere generata da tools integrati nell'ambiente, come Keytool e Jarsigner
- Finche' l'applicazione e' in fase di sviluppo e testing, e' possibile utilizzare una modalita' di firma debug mode
 - In debug mode la firma e' costante (ma l'applicazione non puo' essere pubblicata)
 - Keystore name: "debug.keystore"
 - Keystore password: "android"
 - Key alias: "androiddebugkey"
 - Key password: "android"
 - CN: "CN=Android Debug,O=Android,C=US«"
 - Per passare in release mode e' necessaria una firma dell'utente
- Se si vogliono rilasciare degli aggiornamenti dell'applicazione, e' necessario riutilizzare la stessa firma

Android manifest

- Tutte le caratteristiche esterne di una applicazione Android sono strutturate in un file manifest.xml.
- Manifest.xml e' un file pubblico, che può essere letto in chiaro da ogni possibile utente della applicazione.
- Si tratta di una pratica molto diffusa, nei framework di nuova generazione: dichiarare tutte le costanti di configurazione in file xml statici, che vengono elaborati da qualche metodo del framework, in maniera trasparente al programmatore.

Android manifest: esempio

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.sample.simcareer"
    android:versionCode="1"
    android:versionName="1.0">

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".Landing" android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

componenti base

- **android.app**: implements the Application model for Android
- **android.content**: implements the concept of Content providers
- **android.content.pm**: Package manager: permissions, installed {packages, services, provider, applications, components}
- **android.content.res**: Access to resources
- **android.provider**: Contacts, MediaStore, Browser, Setting

componenti base della GUI

- **android.view**: Menu, View, ViewGroup + listeners
- **android.view.animation**
- **android.view.inputmethod**: Input methods framework
- **android.widget**: UI controls derived from View (Button, Checkbox...)
- **android.gesture**: create, recognize, load and save gestures

grafica

- **android.graphics:**
 - low level graphics tools such as canvases, color filters, points, and rectangles that let you handle drawing to the screen directly.
 - Bitmap, Canvas, Camera (3D transformation, not the camera!) , Color, Matrix, Movie, Paint, Path, Rasterizer, Shader, SweepGradient, Typeface
- **android.graphics.drawable:** variety of visual elements that are intended for display only, such as bitmaps and gradients
- **android.graphics.drawable.shapes**
- **android.opengl:** opengl-related utility classes, not the opengl!
- **javax.microedition.khronos.opengles**
- **javax.microedition.khronos.egl**
- **javax.microedition.khronos.nio**
- **android.renderscript:** low-level, high performance means of carrying out mathematical calculations and 3D graphics rendering

rendering del testo

- **android.text**: classes used to render or track text and text spans on the screen
- **android.text.method**: Classes that monitor or modify keypad input.
- **android.text.style**: Text styling mechanisms
- **android.service.textservice**: Provides classes that allow you to create spell checkers
- **android.view.textservice**: Use spelling checkers

database, web, e location

- **android.database**: classes to explore data returned through a content provider.
- **android.database.sqlite**: the SQLite database management classes that an application would use to manage its own private database. Applications use these classes to manage private databases.
- **android.webkit**: tools for browsing the web.
- **android.location**: Address, Geocoder, Location, LocationManager, LocationProvider
- **com.google.android.maps**

rete e telefonia

- **android.net**: Socket-level network API - help with network access, beyond the normal java.net.* APIs.
- **android.net.wifi**
- **android.bluetooth**
- **android.nfc**: Near Field Communication (NFC) is a set of short-range wireless technologies, typically requiring a distance of 4cm or less to initiate a connection. NFC allows you to share small payloads of data between an NFC tag and an Androidpowered device, or between two Android-powered devices.
- **android.telephony**:
 - monitoring the basic phone information, plus utilities for manipulating phone number strings, SMS
 - CellLocation, PhoneNumberUtils, TelephonyManager
- **android.telephony.gsm**: Obtain Cell location of GSM
- **android.telephony.cdma**: Obtain Cell location of CDMA - CDMA2000 is a family of 3G mobile technology standards

media e speech

- **android.media:**
 - manage various media interfaces in audio and video
 - MediaPlayer, MediaRecorder, Ringtone, AudioManager, FaceDetector.
- **android.media.effect:** apply a variety of visual effects to images and videos
- **android.hardware:** support for hardware features, such as the camera and other sensors
- **android.drm:** Digital right management
- **android.mtp:** interact directly with connected cameras and other devices, using the PTP (Picture Transfer Protocol)
- **android.speech:** base class for recognition service implementations
- **android.speech.tts:** Text to Speech

utilities generali

- **android.util:** date/time manipulation, base64 encoders and decoders, string and number conversion methods, and XML utilities.
- **android.sax:** XML parsing
- **android.test:** A framework for writing Android test cases and suites
- **android.preference:** manage application preferences and implement the preferences UI. Using these ensures that all the preferences within each application are maintained in the same manner and the user experience is consistent with that of the system and other applications
- **android.os:**
 - basic operating system services, message passing, and inter-process communication
 - Binder (ipc), FileObserver (changes in files) Handler e Looper (for dealing with message threads), BatteryManager, PowerManager

Android market

- Strumento standard fornito da Android per la pubblicazione e la pubblicizzazione delle applicazioni
- La registrazione è molto semplice e richiede
 - Un Account Google
 - Una quota di iscrizione di 25\$ (~18€) una tantum
- Nel caso in cui si voglia vendere applicazioni bisogna fornire poi anche le proprie coordinate bancarie e il proprio codice fiscale

ricerca di un'applicazione sul market

- Android Play Store e' disponibile
 - come applicazione Web: <https://play.google.com/store>
 - come applicazione client su tutti i dispositivi Android
 - Non disponibile sugli emulatori

- In passato si chiamava Android Market ed era utilizzato solo per la distribuzione di applicazione. Ora, invece, contiene anche contenuti multimediali (film, musica, libri, giornali)

- In alternativa, è possibile distribuire applicazioni proprie anche in maniera diretta
 - Esistono ulteriori market, non approvati da Google

pubblicazione di un'applicazione sul market

- Per pubblicare un'applicazione e' necessario
 - L'apk firmato
 - Almeno 2 screenshots
 - Un'icona ad alta risoluzione e un banner
 - Titolo e descrizione (per ogni lingua che si vuole supportare)
 - Descrizione delle ultime modifiche
 - Tipo e categoria
 - Classificazione dei contenuti
 - Prezzo (se non gratis)

- Sono automaticamente ricavati dal manifest
 - Versioni di Android compatibili
 - Dispositivi compatibili
 - Permessi da richiedere all'utente installatore

cenno ai modelli di business

- Applicazione gratuita
 - E' possibile, pero', entro certi limiti, lucrare dal servizio svolto dall'applicazione, ad esempio con e-commerce
- Applicazione con banner pubblicitari
 - Ogni scaricamento del banner e, in misura maggiore, ogni click su esso fornisce ricavi
 - I ricavi vengono accreditati periodicamente
 - Esistono molti circuiti pubblicitari, tra cui Google Ads
- Applicazione a pagamento
 - Google trattiene il 30% del costo, ma accredita velocemente il ricavato
- Applicazioni con acquisti in-app
 - E' possibile fare acquisti internamente alla applicazione
 - Ad esempio, in un gioco e' possibile acquistare nuove armi, in whatsapp e' possibile acquistare un prolungamento della licenza

application framework

- Il livello di application framework fornisce un insieme di classi, interfacce e package tramite i quali è possibile sviluppare applicazioni. Ad esempio:
 - Activity Manager gestisce il ciclo di vita delle activity
 - Content providers consente la condivisione delle informazioni tra diverse applicazioni e servizi
 - Telephony Manager gestisce le azioni legate al telefono
 - Location Manager gestisce le informazioni legate al GPS
 - Notification Manager consente la gestione delle informazioni visualizzate come alert sulla barra di stato del dispositivo
 - ...

classificazione dei processi

- Processi in «**foreground**»: sono quelli che interagiscono direttamente o indirettamente con l'utente. Stiamo parlando delle applicazioni che, ad esempio, contengono l'Activity attualmente utilizzata o i Service ad essa collegati. Questi sono i processi che Android tenterà di preservare maggiormente. Importante notare che, comunque, anche le applicazioni in foreground non sono del tutto al sicuro. Se ad esempio il sistema non disponesse di risorse sufficienti a mantenerli tutti in vita, si troverebbe costretto ad arrestarne qualcuno.
- Processi **visibili**: non sono importanti come quelli in foreground ma vengono anch'essi grandemente tutelati da Android. Infatti, avendo componenti ancora visibili all'utente anche se non vi interagiscono più, svolgono comunque un ruolo particolarmente critico. Anche in questo caso si tratta di Activity visibili e Service ad esse collegati.
- Processi «**service**»: contengono dei service in esecuzione che generalmente svolgono lavori molto utili all'utente anche se non direttamente collegati con ciò che l'utente vede nel display. Il loro livello di priorità può essere considerato medio: importanti sì ma non tanto quanto le due tipologie di processi precedenti.
- Processi «**cached**»: sono l'ultima categoria e rappresentano processi che non servono più a nulla. Tra questi vengono selezionati quelli da abbattere e finché il sistema operativo non affronta una fase di difficoltà non ha bisogno di eliminare processi di altre categorie se non di questa. I cached process eliminati non rischiano di impattare minimamente sulla user experience e spesso contengono Activity che non sono più visibili.

| le fondamenta di Android

- Activity
- Service
- Content Provider
- Broadcast Receiver
- Intent